CSA2001 – Fundamentals in AI and ML

Project Report

Name	Aditya Kundliya
Registration No.	25BAI10634
Branch	B.Tech CSE (AI-ML)
Year	First Year (2025–26)
Course Code	CSA2001
Course Title	Fundamentals in AI and ML
Date	29 March 2026

VIT Bhopal University
VITyarthi E-Learning Platform

GameMatch AI: A Spec-Aware Game Recommendation System

INTRODUCTION:

GameMatch AI is a sophisticated Command Line Interface (CLI) application designed to bridge the gap between hardware capabilities and gaming experiences. By evaluating a user's PC specifications—specifically RAM, CPU, and GPU—the system provides a curated list of game recommendations that are predicted to run efficiently on the given hardware. The application leverages real-time data from the RAWG Video Games Database API to ensure up-to-date information on ratings, release years, and genres.

MOTIVATION OF THE PROJECT:

The primary motivation behind this project is the increasing complexity of PC hardware configurations. Many gamers struggle to determine if a modern title will perform well on their specific build without first purchasing or downloading the game. This project aims to provide a lightweight, "intelligence-lite" solution that uses weighted scoring logic to offer instant performance predictions and genre-based suggestions.

PROBLEM STATEMENT:

Gamers often face "performance anxiety" when choosing new titles, as official "Minimum" and "Recommended" specs can be vague or misleading. There is a lack of simple, centralized tools that can simultaneously filter games by personal preference (genre) and hardware compatibility without requiring heavy GUI-based software or manual research across multiple databases.

OBJECTIVE OF THE PROJECT:

- To develop a functional CLI-based tool for game discovery.
- To implement a System Power Score algorithm based on weighted hardware components.
- To categorize game performance into four distinct levels: "Runs great," "Runs well," "Runs OK," and "May struggle".
- To integrate external APIs for real-time data retrieval.

EXISTING METHODS:

- **Manual Comparison:** Users manually compare their specs against Steam or Epic Games Store requirement lists.
- **Web-Based Benchmarks:** Sites like "Can You Run It" or "UserBenchmark" offer hardware analysis but often require installing browser plugins or searching through cluttered interfaces.

PROS AND CONS OF THE STATED METHODS:

Method	Pros	Cons
Manual Comparison	No software needed.	Time-consuming; prone to human error; difficult for non-technical users.
Web-Based Benchmarks	Highly detailed data.	Often requires third-party installs; contains heavy advertising; doesn't always suggest new games.

HARDWARE AND SOFTWARE REQUIREMENTS:

- Operating System: Windows/Linux/macOS.
- Language: Python 3.12.
- Libraries: requests (for API calls).
- Internet Connection: Required for RAWG API integration.
- Developer Tools: RAWG API Key.

METHODOLOGY AND GOAL:

The project follows a modular development methodology. It separates the user interface (cli.py) from the processing logic (utils.py). The core goal is to simulate an AI-driven decision process where hardware variables are mapped to a numerical "Tier," which is then compared against the "Demand Tier" of a game based on its release year.

FUNCTIONAL MODULE DESIGN AND ANALYSIS:

- **Input Module:** Captures RAM, CPU, GPU, and preferred Genre from the user.
- **Tiering Engine:** Converts raw strings (e.g., "RTX 3060") into numerical scores using pre-defined dictionaries and keyword parsing.
- **API Client:** Construct dynamic URLs to fetch high-rated games (Metacritic > 60) from the RAWG database.
- **Performance Predictor:** Calculates the delta between the user's system score and the game's release-year demand.

ALGORITHM DEVELOPMENT:

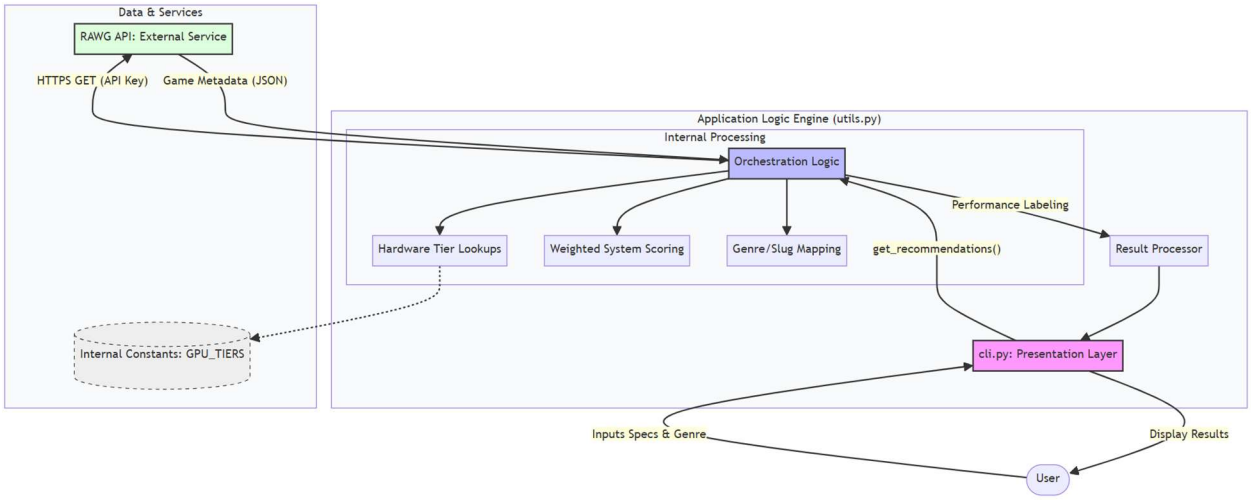
The system computes the **System Power Score** (S) using the following weighted formula:

$$S = (GPU_{tier} \backslash 0.5) + (CPU_{tier} \backslash 0.3) + (RAM_{tier} \backslash 0.2)$$

The performance label is then determined by the difference between S and the Demand score:

- **Difference >= 3:** Runs great.
- **Difference >= 1:** Runs well.
- **Difference >= 0:** Runs OK.
- **Difference < 0:** May struggle.

SOFTWARE ARCHITECTURAL DIAGRAM:



CODING:

The implementation is split into two primary files:

- cli.py: Handles the input() loop and result printing.
- utils.py: Contains GPU_TIERS, ram_tier(), cpu_tier(), and the get_recommendations() function.

OUTPUT:

The system outputs a formatted list of games.
Example:

```
🔥 Recommendations:

🎮 Half-Life
   Half-Life (1998) - Runs great
   ⭐ 4.4/5 | 1998

🎮 Red Dead Redemption 2
   Red Dead Redemption 2 (2018) - May struggle
   ⭐ 4.6/5 | 2018

🎮 Half-Life 2
   Half-Life 2 (2004) - Runs great
   ⭐ 4.5/5 | 2004

🎮 BioShock
   BioShock (2007) - Runs well
   ⭐ 4.4/5 | 2007

🎮 Grand Theft Auto IV: Complete Edition
   Grand Theft Auto IV: Complete Edition (2010) - Runs well
   ⭐ 4.6/5 | 2010

🎮 Grand Theft Auto IV
   Grand Theft Auto IV (2008) - Runs well
   ⭐ 4.3/5 | 2008
```

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM:

- **Weighted Scoring:** Prioritizes GPU as the most critical component for gaming (50% weight).
- **Keyword Matching:** Uses fuzzy string matching to identify CPU and GPU tiers from user input (e.g., finding "i7" within "Intel Core i7-12700K").
- **Genre Mapping:** Maps user-friendly genre names to RAWG-specific API slugs.

SIGNIFICANT PROJECT OUTCOMES:

- Successful integration of a third-party REST API.
- Creation of a functional heuristic model for hardware-software compatibility.
- A lightweight (less than 10KB) application footprint.

TESTING AND REFINEMENT:

- **Input Validation:** The system uses try-except blocks to handle non-numeric RAM inputs.
- **Fallback Logic:** If a GPU is not found in the GPU_TIERS dictionary, the system assigns a default mid-range tier of 5 to maintain functionality.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS:

- **E-commerce Integration:** Could be used by retailers to suggest games alongside PC hardware purchases.
- **Game Launchers:** Could be integrated into lightweight launchers (like GOG or itch.io) to provide performance warnings before installation.

FINDINGS OF THE PROJECT:

The project found that release year is a highly effective, albeit simplified, proxy for a game's hardware demand. By tiering games into eras (e.g., pre-2005, 2018-2021), the system can accurately predict performance for the majority of mainstream titles.

LIMITATIONS OF THE SYSTEM:

- **Static Hardware List:** The GPU_TIERS dictionary must be manually updated for new releases.
- **Optimization Variations:** The system cannot account for poorly optimized games that may lag even on high-end hardware.
- **Connectivity:** Requires an active internet connection for API data.

CONCLUSIONS:

GameMatch AI demonstrates that basic AI/ML concepts like weighted scoring and classification can be applied to build practical, user-centric tools. It successfully fulfills the requirements for a modular, CLI-based recommendation system for the CSA2001 course.

FUTURE ENHANCEMENTS:

- **Machine Learning Integration:** Use a regression model trained on actual benchmark data to replace the manual tiering system.
- **GUI Development:** Transition from CLI to a modern GUI using Tkinter or PyQt.
- **Automatic Spec Detection:** Use the psutil or py-cpuinfo libraries to automatically detect user hardware.

REFERENCES:

- Python Documentation (**requests** library).
- RAWG Video Games Database API Documentation.
- VITyarthi Course Material: Fundamentals in AI and ML (CSA2001).